Run A* and UCS on the same graph

The graph, edge costs, start, and goal stay identical in both runs. UCS is A* with every heuristic value set to zero.

2.1 Results comparison

Measurement	A* (supplied heuristic)	UCS ($h = 0$)
Returned path	S → A → C → G	S → A → C → G
Path cost	$1 + 2 + 3 = 6$	$1 + 2 + 3 = 6$
Processed-node trace	S, A, C, G	S, A, C, B, G
Expanded nodes	S, A, C	S, A, C, B
Expansion count	3	4

A* processed-node calculations

Node	g	h	f
S	0	5	5
A	$0 + 1 = 1$	4	5

C	$1 + 2 = 3$	2	5
G	$3 + 3 = 6$	0	6

Returned path cost:

\[1+2+3=\boxed{6} \]

UCS processed-node trace

UCS uses $f(n) = g(n) + 0 = g(n)$. After expanding C, it has a cost-6 route to G, while B has a cost of 4. Therefore, B is processed before G.

Node	**g**	**h**	**f**
S	0	0	0
A	1	0	1
C	3	0	3
B	4	0	4
G	6	0	6

The route through B costs:

$$4 + 5 = 9$$

But the route through A and C costs:

\[1+2+3=\boxed{6} \]

UCS therefore returns the cheaper route through A and C.

How are nodes counted?

Following the manual's convention:

- A processed node is a valid entry removed from the priority queue.
- An expanded node is a non-goal node whose successors are examined.
- The goal G is included in the processed trace but not in the expansion count.
- Stale queue entries are ignored.

In these runs, the cost-9 goal entry is never processed, because the cost-6 goal entry is selected first. No stale entries are skipped in the recorded runs.

Conclusion: A* and UCS both find the optimal path with cost 6. On this graph, A* expands 3 nodes while UCS expands 4, so A* explores fewer nodes.